THOMPSON RIVERS UNIVERSITY

SENG 4640 - Software Engn for Web Apps

Building a Scalable E-commerce Web Application

Junepng Jiang (T00657653)

Marium Tawhid (T00585209)

Date submitted

Table of Contents

1 Executive Summary 6

2 Introduction 7

3 Design Problem 7

3.1 Problem Definition 7

3.2 Design Requirements 7

3.2.1 Functions 7

3.2.2 Objectives 7

3.2.3 Constraints 7

4 Solution Development 8

4.1 Solution 1 8

4.2 Solution 2 8

4.3 Final Solution 8

4.4 Components & Features 8

5 Prototype Development and Testing 9

5.1 Initial Testing 9

5.2 Report on Final Testing 9

5.3 Environmental, Societal, Safety, and Economic Considerations 9

5.3.1 Safety Analysis - to identify possible hazards in the design and evaluate the risk that could lead to an accident. 9

5.3.2 Economic Considerations 9

5.3.3 Life Cycle and Environmental Impact Analysis 10

5.4 Limitations 10

6 Team Work 11

6.1 Meeting 1 11

6.2 Meeting 2 11

6.3 Meeting 3 11

6.4 Meeting 4 11

7 Project Management 13

8 Conclusions 14

10 References 15

- The table of contents should be automatically generated by selecting "References/ Table of Contents". Remember that the table of contents should not have an entry of the "Table of Contents" itself.
- Proof read the text for typing and grammar mistakes.
- Follow the IEEE Bibliography style for the references by selecting "References/ Citations & Bibliography/ Style".
- See the guide in the link for more details:
- https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf

List of Figures

- Similar to table of contents, list all figures in the report with page numbers.

List of Tables

- Similar to table of contents, list all tables in the report with page numbers.

1 Executive Summary

The summary is considered by many to be the most important part of a report. It is a difficult part to write and should always be written last. It is often the only part that managers in the work place have time to read, so it is important to state your case succinctly (normally less than one page).

Purpose: Provides a brief summary of the report, an information abstract. No reference is made to any part of the report; a summary is complete in itself.

Content:

- states the more important information in the report
- the purpose, method, reason for the report
- the main findings, conclusions, decisions, and recommendations
- all major generalizations or assumptions of the report
- There shouldn't be any references in the executive summary. The research and background go in the introduction of this report.

Note about the report: The length of your report and each section should reflect the complexity of the topic and the thoroughness of the research and design. The report should be self-contained (any terminology particular to the topic should be defined the first time you use it), consistent, and to the point. It should be understandable to someone who has background in the area of the report but is unfamiliar with the particular topic of the report. Use standard formal level of English (no slang or colloquialisms)

2 Introduction

- Introduces the subject of the report to the reader. Remember that the reader may be from a different branch of the discipline, and will require some orientation to the subject of your report.
    - Note that for this report, there is no background section, so the background is combined with the introduction. Start the introduction section by showing some research and background on the topic of your subject. Make sure that you include references for everything in the report that is not your own thoughts, and that everything is referenced in IEEE style. For IEEE, put a number (i.e. [1]) next to the referenced phrase or figure title, and then be sure to state the reference in your reference section. Microsoft word can do this for you- use “references” tab, then “insert citation”.
- Should contain the subject and purpose of the report: states briefly why the report is being written and what the re- port is intended to achieve.
- Scope: describes how broad or how limited the treatment of the subject will be
- Plan of development: outlines which areas will be covered
- Thesis of the report: the general conclusion and/or the general recommendation

The rapid growth of e-commerce has transformed the way consumers interact with digital marketplaces, creating a demand for scalable, secure, and user-centric web applications. Platforms such as Amazon and eBay have set high expectations for seamless shopping experiences, real-time inventory management, and personalized user interactions. As part of the SENG 4640 course, this project aims to design and develop a fully functional e-commerce web application that emulates the core features of modern online retail systems while addressing key engineering challenges such as concurrency control, data integrity, and scalable architecture.

The primary objective of this project is to build a web application that supports multiple user roles—including Guests, Customers, Product Managers, and Super Admins—each with distinct permissions and capabilities. The system will enable product browsing, shopping cart management, order processing, and administrative controls. A central technical challenge is the implementation of a high-concurrency "Flash Sale" feature, which requires robust architectural strategies to prevent inventory overselling under simultaneous user requests.

To meet these goals, the project will follow a structured engineering design process, beginning with requirement analysis and progressing through architecture selection, prototyping, testing, and deployment. The final deliverable will be a cloud-deployed application accessible via a public URL, accompanied by a technical report and video demonstration. This report documents the design decisions, testing methodologies, and outcomes, with a focus on measurable performance metrics and validation of the concurrency control mechanism.

By completing this project, the team aims to demonstrate proficiency in full-stack web development, system architecture design, and engineering best practices, while also

considering broader impacts such as data privacy, accessibility, and environmental sustainability.

3 Design Problem

Open each section of the report with an introductory paragraph (at least a few sentences) on what the section is about.

This section has the following two subsections:

3.1 Problem Definition

Write a problem statement of the project.

Describes the “gap” (i.e., need) that you are seeking to fulfill with the solution to the problem.

The modern e-commerce landscape demands web applications that are not only feature-rich and user-friendly but also capable of handling high-traffic events with precision and reliability. Consumers expect seamless browsing, secure transactions, and real-time inventory accuracy, while administrators require robust tools for product management and user oversight. However, developing such a system presents significant engineering challenges, particularly when simulating time-sensitive events like flash sales, where multiple users may attempt to purchase limited-stock items simultaneously. Without proper concurrency control, these scenarios can lead to inventory overselling, data inconsistency, and poor user experience.

Currently, there is a need for a scalable e-commerce platform that supports multiple user roles, dynamic product catalogs, shopping cart functionality, and secure order processing. Additionally, the system must demonstrate resilience under high-concurrency conditions, such as during flash sales, by employing deliberate architectural strategies to prevent overselling. This project addresses this gap by designing and implementing a full-stack e-commerce web application that meets these functional and non-functional requirements while adhering to industry standards for performance, security, and accessibility.

3.2 Design Requirements

This section has the following two subsections:

3.2.1 Objectives

- Provide objectives of the design project. Remember that the objectives are specified as adjectives.
- Include an Objective tree. Remember that objectives are attributes that the final design should possess- such as safe, economical, portable, etc. Each primary object should have more detailed objectives below it (sub-objectives).

The design objectives represent the desirable attributes that the final system should possess. These are organized into a hierarchical objective tree to illustrate the relationships between high-level goals and their sub-objectives.

Primary Objectives:

Functional: The system shall support all core e-commerce operations, including user management, product browsing, cart handling, and order processing.

Secure: The system shall protect user data and prevent unauthorized access through encryption, authentication, and adherence to data protection regulations.

User-Centric: The system shall provide an intuitive, responsive, and accessible interface that caters to diverse user needs and device types.

Scalable: The system architecture shall support growth in user base and product inventory without degradation in performance.

Reliable: The system shall maintain data integrity and availability, especially during high-concurrency events like flash sales.

Testable: The system shall include automated testing for critical functions to ensure correctness and robustness.

3.2.2 Constraints

- Provide constraints here. Remember that the constraints are binary (either satisfied or not). Constraints are used to verify if a solution can be considered or discarded.

The following constraints are binary conditions that any viable solution must satisfy. These are derived from the project requirements and must be met for a design to be considered acceptable.

User Role Support: The system must support four distinct user roles: Guest, Customer, Product Manager, and Super Admin, each with appropriate permissions as specified.

Flash Sale Concurrency Control: The system must guarantee zero inventory overselling during simultaneous purchase requests in flash sale events, using a deliberate concurrency control mechanism (e.g., locking, queuing, optimistic concurrency).

Product Catalogue Size: The system must be populated with at least 50 distinct products across multiple categories, generated via seed scripts if necessary.

Deployment: The system must be deployed to a live, publicly accessible cloud environment (e.g., AWS, Render, Google Cloud) and cannot be restricted to localhost.

Automated Testing: The system must include automated unit tests for at least three critical backend functions (e.g., authentication, order calculation, inventory management).

Load Testing Validation: The flash sale mechanism must be validated using automated load-testing tools (e.g., JMeter) to demonstrate stability under high concurrency.

Data Protection Compliance: The system must comply with data protection regulations such as GDPR or CCPA, including user consent mechanisms and data transparency.

Accessibility Standards: The system must adhere to WCAG accessibility guidelines, including keyboard navigation, alternative text for images, and sufficient color contrast.

Security: The system must be resilient to common web vulnerabilities, including XSS and SQL injection, and must encrypt sensitive data (e.g., passwords, payment details).

Payment Support: The system must support at least one payment option (e.g., credit card or PayPal) during checkout.

4 Solution Development

- Provide multiple, valid, different solutions to the problem. Each design should represent a different approach to the problem, a different set of trade-offs between the objectives.
- Enable the reader to have a clear picture of each alternative as well as its strengths and weaknesses in order to compare them.
- All alternatives must be feasible.
    - There should not be one clearly superior design and several clearly inferior designs,
    - nor should the alternative designs be merely components of a single design that emerges as the proposed design in the end.

4.1 Solution 1

Gives, in one or two paragraphs, a short description of the alternative design.

Reviews, in bullet lists, advantages and disadvantages with respect to project requirements, such as objectives and constraints.

Provides some preliminary sketches, drawings, and/or diagrams (e.g., flowchart or use case diagram) of the design, with measurements where appropriate; extra figures may be put in the appendices. Refer your reader to these appendices clearly in the body of the report, identifying each appendix by alphabetical designation and title (e.g., Appendix A: Design Option 1—Details of the Power Supply).

Uses parallel structure to make it easier for the reader to understand the comparison. Parallel structure means structuring the description of each design alternative the same way, with information given in the same order using the same formatting.

*Below each figure, figures should be numbered and labelled as Figure 1, 2, etc. followed by the figure title.

*Tables should be numbered and labeled above each table as Table 1, 2 etc., followed by the table title.

**Be sure that before a figure or table is placed in the report, there is a discussion about the figure or table, and what it shows. (Don't just plop figures or tables into the report without out some introduction/commentary about them first).

The first solution is a minimal frontend prototype built directly from the provided Home.html and Home.css files. React and ReactDOM are loaded via CDN <script> tags, and Babel is included to enable JSX compilation directly in the browser. All React components (Navbar, Banner, ProductCard, ProductList, App) are written inside a single <script type="text/babel"> block within the HTML file.

Product data is hard-coded as a static array. The CSS styles are placed in a separate Home.css file and linked to the HTML. The page displays a navigation bar, a promotional banner, and a grid of product cards. Clicking the “Add to Cart” button triggers a simple JavaScript alert() to simulate adding an item to the cart. There is no backend, no persistence, and no user authentication.

Advantages

- Zero setup: No build tools or local development environment required; simply open the HTML file in a browser.
- Fast prototyping: Changes can be made directly in the HTML/CSS and instantly viewed after a page refresh.
- Small footprint: All code is contained in two files, making it easy to share or demonstrate the basic concept.
- Demonstrates core React concepts: Component composition, props, and JSX are used in a straightforward manner.
- Improved Code Maintainability: having a modular structure means that each user interface element is encapsulated in it’s own component

Disadvantages

- Not scalable: All data is static and hard-coded; any update requires editing the HTML file.
- No backend integration: Cannot support user accounts, real product management, or order processing.
- Poor performance: Babel compilation in the browser slows down initial page load, and React is loaded from a CDN every time.
- No concurrency handling: The flash sale requirement cannot be addressed because inventory is not shared.
- Not maintainable: As the application grows, mixing all components in one file becomes unmanageable; there is no module system.
- Security concerns: No authentication, encryption, or protection against common web vulnerabilities.

This solution serves as a proof of concept for the user interface and establishes a baseline for more advanced iterations. It meets the visual requirements of the project but fails to satisfy most functional and non-functional requirements.

4.2 Solution 2

As above for Solution 1, but for a second alternative design.

The second solution evolves the frontend into a modern, modular React application created with Create React App or Vite. The codebase is organized into reusable components (e.g., ProductCard, Navbar, Cart), and state is managed using React hooks or a state management library like Redux Toolkit. The application uses mock data (static JSON files) to simulate products and cart operations. Routing is implemented with React Router to enable multiple pages (home,

product details, cart, checkout). However, there is still no backend – all operations are performed in the browser and do not persist across sessions. This solution serves as a fully functional frontend prototype that closely mimics the look and feel of the final product.

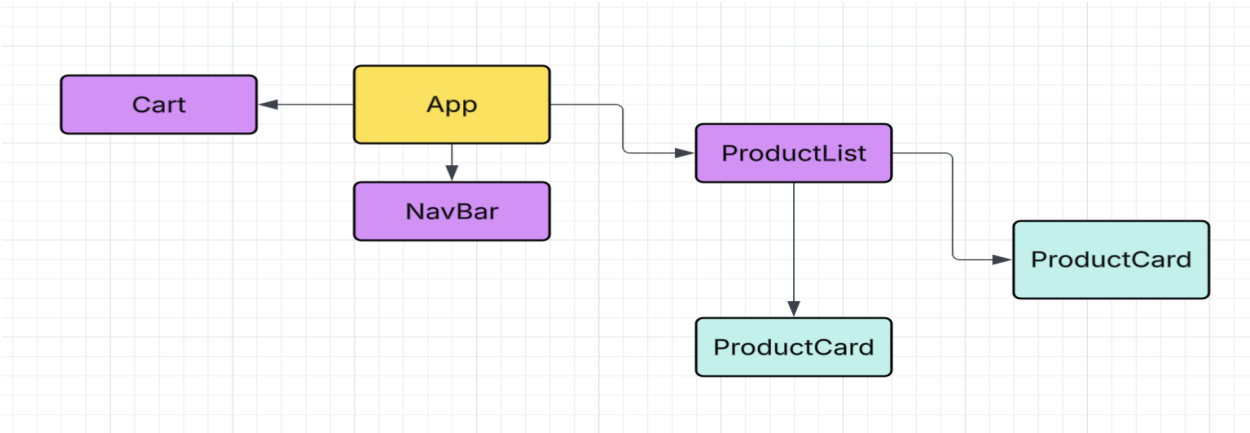

Figure 1: React Component hierarchy for the modular frontend architecture

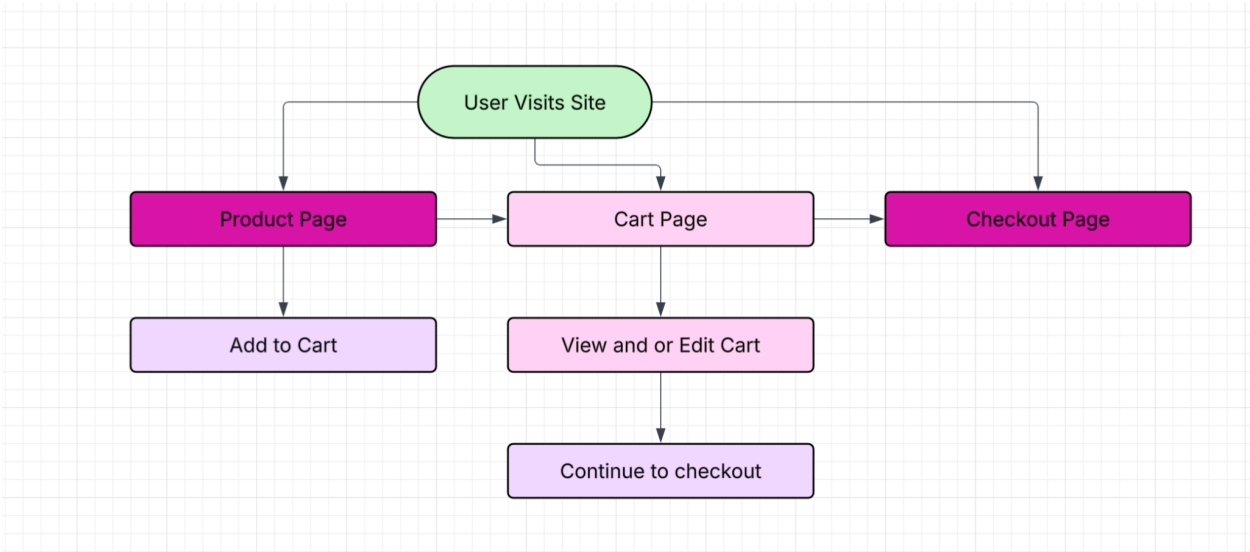

Figure 2: ROuting structure on client side using React.

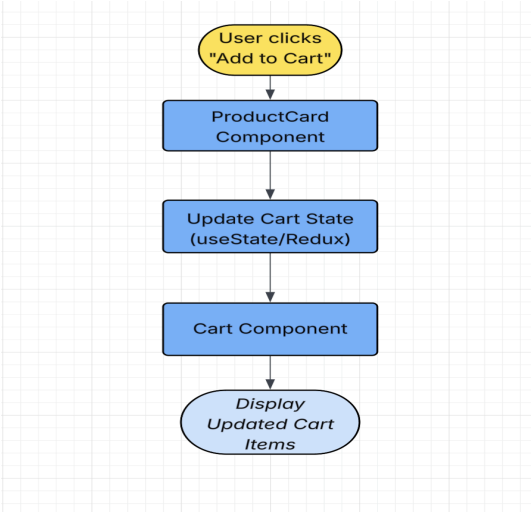

Figure 3: State management workflow for updating shopping carts.

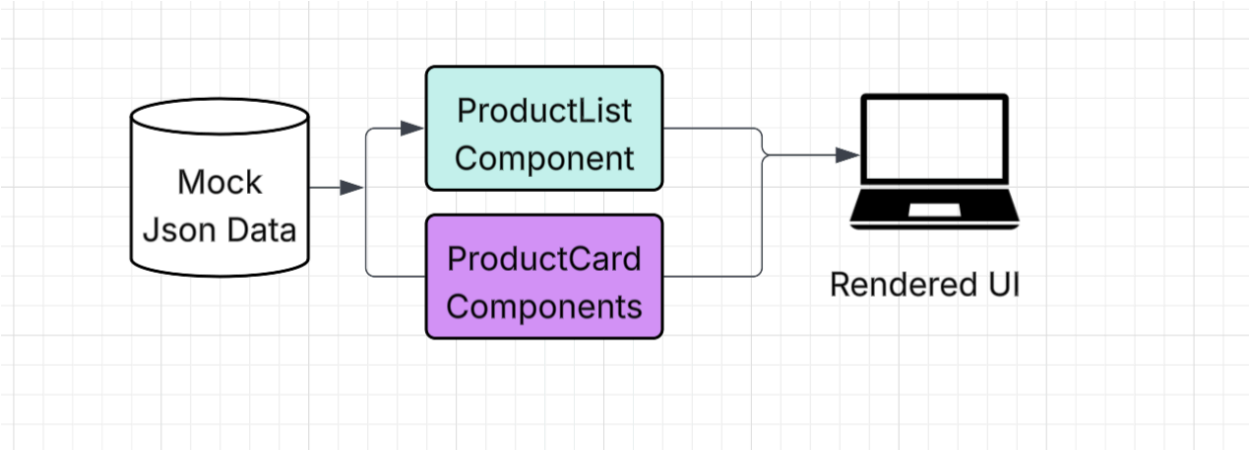

Figure 4: Product Data flow from the mock json files to the UI components

Advantages

- Modular and maintainable: Components, clear separation of concerns, and modern tooling.
- Better performance: Build process optimizes code (minification, tree shaking) and supports code splitting.
- Realistic user experience: Routing, state management, and responsive design can be fully implemented.
- Easy to extend: Adding a backend later requires only replacing mock API calls with real HTTP requests.

Disadvantages

- No real user authentication: User roles (guest, customer, admin) cannot be enforced since all the logic runs on the client environment, which can be easily manipulated by malicious people.
- Cannot handle concurrency: Flash sales cannot be tested because inventory is not shared. This is a major requirement. Since solution 2 stores product inventory locally in the browser, each user operates on an isolated dataset -> prevents the system from being able to coordinate parallel purchase requests -> not reliable inventory management.

- No data protection or security measures: The application does not address real-world security concerns. Without a server-side validation , malicious users can easily manipulate browser data -> manipulate carts and pricing -> not reliable and unsuitable for handling operations that work with payment processing.

- No persistent data storage: The shopping cart content and any product interactions disappear when you refresh the page. This is because the system operates entirely on the client side, so once the page refreshes, all data is lost -> not a realistic user story for a shopping workflow.

Below is a table that compares Solution 1 and Solution 2 on 5 features: Architecture, State Management, Scalability, Data Source, and Routing. As can be observed, Solution 2 does improve on some of the limitations of Solution 1, but not quite meeting the requirements of the project.

Feature	Solution 1	Solution 2
Architecture	Single Html file	Modular React components
State Management	None	React hooks and Redux
Scalability	Very, very limited	Moderate, can add components.

Data Source	Hard-coded array	Mock Json API
Routing	None	React Router

Table 1: Comparison between Solution 1 and Solution 2.

4.3 Final Solution

This is the final solution. Explain why it is better than other solutions. You may use a table for comparison purposes.

The third solution builds upon the modular React frontend from Solution 2 by adding a complete backend and a message queue to address the core engineering challenge: the high-concurrency flash sale. The technology stack is the MERN stack (MongoDB, Express, React, Node.js) extended with Redis as a message broker, all deployed on a cloud infrastructure.

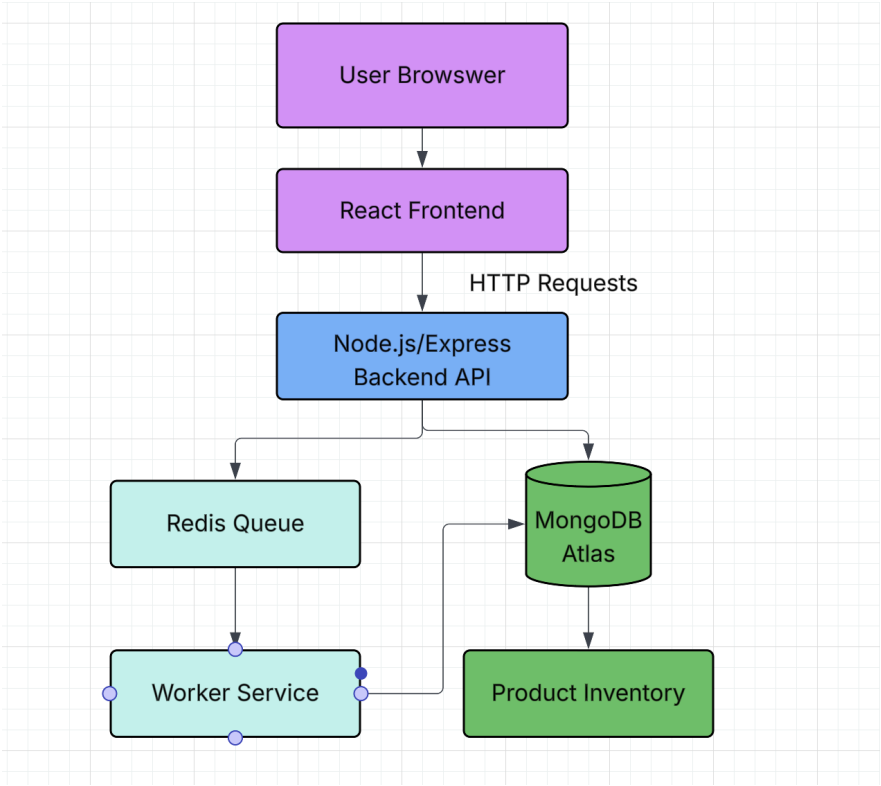

Figure 5: Cloud-based Architecture for the final e-commerce system

- **Frontend**: A Vite-built React application hosted on Firebase Hosting or Google Cloud Storage. Vite improves the development performance and produces builds that can be used for deployment.
- **Backend API**: A Node.js/Express API deployed on Google Cloud Run (or Compute Engine) for automatic scaling.
- **Database**: MongoDB Atlas, a fully managed cloud database, ensuring high availability and data durability. It stores persistent application data such as user accounts, inventory counts, product information, and orders.
- **Message Queue**: Redis (hosted on Google Cloud Memorystore or a managed Redis service) to decouple purchase requests from inventory updates.

To prevent overselling during flash sales, purchase requests are not processed synchronously. Instead:

1. The frontend sends a purchase request to the API.
2. The API creates a job in the Redis queue and immediately returns a jobId to the frontend.
3. A separate worker service consumes jobs sequentially (or with per-product locking), checks inventory in MongoDB Atlas, and updates the stock atomically.
4. The frontend polls a status endpoint to inform the user of the outcome (success or failure).

This architecture guarantees zero overselling under any load while providing a responsive user experience.

Advantages :

- Guaranteed no overselling: The queue serialises access to each product, eliminating race conditions.
- High scalability: Redis queue absorbs traffic spikes; workers can be horizontally scaled, adding servers and workers as needed to handle higher traffic. It also ensures that purchase requests are processed sequentially vs simultaneously which prevents overselling on nonexistent products.
- Cloud-native and accessible: Deployed on Google Cloud with MongoDB Atlas, meeting the public URL requirement.
- Excellent user experience: Users receive immediate acknowledgment and can track order status via polling.
- Secure and compliant: Implements authentication, encryption, GDPR consent, and input validation.This protects the app from common vulnerabilities like SQL injection, or cross-site scripting.
- Persistent data storage: All data is stored in the cloud, ensuring no loss on page refresh.
- Testable and verifiable: Supports automated unit tests and load tests (e.g., JMeter) to validate concurrency handling.

- Realistic user roles: Server-side enforcement of permissions for Guest, Customer, Product Manager, and Super Admin.

Disadvantages

- Increased complexity: Requires managing Redis, asynchronous workflows, and cloud services. Managing these components requires much more configuration and monitoring.
- Additional development effort: Frontend needs polling (or WebSockets) to show order status.
- Potential cost: Cloud services may incur small charges, but free tiers are sufficient for the project scope.
- Learning curve: The team must become familiar with Google Cloud and MongoDB Atlas, and they must also be family with asynchronous processing, distributed systems, and cloud deployment strategies in order to add , manage, and configure the extra components.

Why Solution 3 is the Best Choice

- It is the only solution that meets all project requirements, particularly the critical high-concurrency flash sale constraint.
- The queue-based concurrency control guarantees data integrity and prevents overselling, which is impossible in Solutions 1 and 2.
- Deployment on Google Cloud and MongoDB Atlas provides a publicly accessible, scalable, and production-ready environment, satisfying the “public URL” mandate.
- The modular frontend from Solution 2 can be reused, preserving previous development effort whilst at the same time ensuring that the system exhibits fast development performance and builds.
- The architecture inherently supports security, testing, and future enhancements (e.g., adding payment gateways, analytics).
- The separation of frontend and backend allows for each of the services to be independently scaled dependent on the size of traffic, rather than being tightly coupled.

- Therefore, Solution 3 is selected as the final architecture. The following sections detail its implementation, testing, and deployment.

Feature	Solution 1	Solution 2	Solution 3
Architecture	Static Html prototype	Modular React Spa	Distributed MERN architecture
Backend	None	None	Node.js/ Express API
Database	None	Mock JSON	MongoDB Atlas
Flash Sale Support	impossible given structure	Impossible	Guaranteed
Scalability	Extremely Limited	Moderate due to modular structure	High
Deployment	Local file	Local build	Cloud deployment
Data Persistence	None	Session by session only/isolated	Cloud Database
Security	None	Client-side only	Authentication and Validation
Concurrency Handling	Not supported	Not supported	Redis queue

Table 2: Comparison of all three solutions

4.4 Components & Features

Describe each of the components and features for each of the parts of the final design in detail.

Refer to SolidWorks diagrams as each component is described.

Use the correct engineering terminology and describe the physical process and energy transformations that are powering the motion in each step. For example, chemical energy in a battery is transformed to electrical energy and then to mechanical energy to turn something. Types of energy include, but are not limited to: gravitational, kinetic (the energy an object has due to it's motion), elastic, thermal, sound, chemical, and electrical.

The final system architecture is composed of several interconnected components that work together to create a functional, e-commerce platform. Each component in the system has a designated responsibility, which enables modular development and scalability. The e-commerce system is divided into five primary component layers:

- Frontend Interface

- API service layer
- Category Component Modules
- Data Storage Layer
- Concurrency Control Layer

4.4.1 Frontend Interface Component

4.4.2. Category-Based Components

5 Prototype Development and Testing

.

5.1 Initial Testing

What have you used to test your design to ensure that it meets the objectives?

What improvements were made during the testing and evaluation of the prototype?

Photographs (figures) of the testing process may be helpful to explain things.

5.2 Report on Final Testing

The purpose of this section is to provide as detailed evidence as possible that the proposed design successfully meets the project requirements. The section is written as a table and indicates:

- How have you tested your proposed design to ensure that it meets the objectives?
- Clear measures of success
- The section can be written as a table preceded by an introductory paragraph.

Photographs of the final testing process can provide useful visualization aids. Be sure to also submit a video of the prototype during the development process.

5.3 Environmental, Societal, Safety, and Economic Considerations

Explain how your engineering design considered environmental, societal and economic considerations. What type of economic decisions did you make? How did you make sure that the design is safe to use, etc.

5.3.1 Safety Analysis - to identify possible hazards in the design and evaluate the risk that could lead to an accident.

- An introductory paragraph
- Hazards of the design, considering how they may be reduced or eliminated or how safer alternatives might be used.
- Accidents and how humans and property may be guarded.
- Warnings and training for the operators

5.3.2 Economic Considerations

- Detailed costs of each of the purchased components and how well the project remained under budget. A table to break this down may be helpful.
- Discuss where the other components came from. This leads into the next section- life cycle analysis.

5.3.3 Life Cycle and Environmental Impact Analysis

- Provide a life-cycle analysis; a 3 R's approach - reduce, reuse and recycle.
- Some background research on the life cycle of the items used for your project would be helpful to show that this was considered in depth. (Be to reference your sources).
- Discuss how the design tries to lessen the negative impacts and increase the positive ones.
- Recommendations for construction/design/implementation/operation/de-commissioning to reduce environmental impact.

5.4 Limitations

Every product has some limitations, and so is the case with your design product. Highlight some of the limitations of your prototype here.

6 Team Work

This section of the report discusses and showcases the collaboration between the team members to effectively coordinate our efforts to complete the task.

Regular meetings were held during the duration of the project to plan tasks, evaluate the different design alternatives, and keep track of progress.

6.1 Meeting 1

Time: February 10th, 2026, 3-4pm.

Present:

- Junpeng Jiang (T00657653)
- Marium Tawhid (T00585209)

Agenda:

- To discuss the project requirements and deliverables.
- Define design problem
- Discuss potential ideas

Meeting minutes:

The team discussed project instructions and deliverables, including the report, a prototype , and a full scaled e-commerce site at the end. The team also discussed potential challenges, current skillsets, and determined areas of weakness (or complete lack of knowledge).

Team Member	Previous Task	Completion State	Next Task
Junpeng	N/A	N/A	Research Scalable Web Architectures
Marium	N/A	N/A	Research existing e-commerce platforms and determine key system features.

Discussion Items:

- Design Direction: The team discussed whether to begin with frontend or backend development. Taking into account the current course material, it was decided to start with frontend development.

Record of team rule penalty: N/A, none were issued during this meeting.

6.2 Meeting 2

Time: February 19th, 2026, 5-6pm

Present:

- Junpeng Jiang (T00657653)
- Marium Tawhid (T00585209)

Agenda:

- Review research findings
- Define system objectives and constraints
- Begin drafting report section
- Discuss solution 1.

Meeting minutes:

During the duration of the meeting, the team went over the research done in the previous work. Key ideas such as best practices for web applications, and methods for structuring frontend applications were highlighted and used as inspiration for solution 1.

Team Member	Previous Task	Completion State	Next Task
Junpeng	Research Scalable Web Architectures	100%	Draft problem definition and design requirements.
Marium	Research existing e-commerce platforms and determine key system features.	100%	Draft solution 1 description , advantages, and disadvantages

Discussion Items:

- After discussion, solution 1 was chosen to be a prototype that showcases user interface, but without any backend functionality.

Record of team rule penalty: Not Applicable

6.3 Meeting 3

Time: February 26th 2026, 7-8pm

Present:

- Junpeng Jiang (T00657653)
- Marium Tawhid (T00585209)

Agenda:

- Review report progress.
- Discuss potential solutions for solution 2.
- Discuss system architectures
- Plan frontend implementation.

Meeting minutes:

The team reviewed the draft for the report and confirmed the first and second section, which included problem definition, objectives, and potential constraints. Potential ideas for solution 2 were discussed, which was then decided to be a reusable components system with components: ProductCard, ProductList, Navbar, and Cart. to improve the maintainability and the scalability of the web application. This architecture would also implement a client side routing (using React Router) and use mock json files to simulate backend data.

Team Member	Previous Task	Completion State	Next Task
Junpeng	Draft problem definition and design requirements.	N/A	Implement React Component structure and routing
Marium	Draft solution 1 description , advantages, and disadvantages	N/A	Write solution 2 section and create system diagrams

Discussion Items:

- Frontend Architecture: The team agreed to use a frontend architecture as it would make it easier to scale when implementing a full stack web application at the end.

Record of team rule penalty: Not Applicable, none where applied.

6.4 Meeting 4

Time: March 4th 2026, 8-9 pm

Present:

- Junpeng Jiang (T00657653)
- Marium Tawhid (T00585209)

Agenda:

- Review prototype progress
- Finalize solution 2 documentation and implementation
- Ensure all deliverable components are ready for the next interim submission.
- Plan final system architecture.

Meeting minutes:

The team reviewed the progress of the frontend prototype and talked about possible improvements to the user interface and architecture (looking at the advantages and disadvantages). It was agreed that the system would expand onto the modular frontend, but also integrating a backend architecture with backend API, a database for product management, authentication, and order processing.

The team also finalized solution 2 section and began planning and assigning roles for solution 3.

Team Member	Previous Task	Completion State	Next Task
Junpeng	Implement React Component structure and routing	100%	Develop Backend API outline and a concurrency strategy.
Marium	Write solution 2 section and create system diagrams	100%	Complete report formatting, diagrams, and testing sections.

Discussion Items:

- Final System Design: the team agreed that the final solution will integrate a modular react frontend with a backend server and database.

6.5 Meeting 5

Time: March 11th 2026, 8-9 pm

Present:

- Junpeng Jiang (T00657653)
- Marium Tawhid (T00585209)

Agenda:

- Review prototype progress
- Finalize solution 3 documentation
- Ensure all deliverable components are ready for the next interim submission.
- Plan to design components of final system.

Meeting minutes:

The team reviewed the progress of the frontend prototype and talked about possible improvements to the user interface. It was agreed that the system would expand onto the modular frontend, but also integrating a backend architecture with backend API, a database for product management through MongoDB altas, authentication, and order processing through Redis Queue and a worker service to avoid missordering.

The team also finalized solution 3 section and began planning and assigning roles for backend implimentation.

Team Member	Previous Task	Completion State	Next Task
Junpeng	Develop Backend API outline and a concurrency strategy.	100%	Develop Backend API from Node.js to worker service.
Marium	Complete report formatting, diagrams, and testing sections.	100%	Design frontend components style.

Discussion Items:

- Final System Design: the team agreed the design of final solution, and plan the Tasks for teammembers in next delieverible.

Record of team rule penalty: Not Applicable

Note in the Last meeting to add a summary of Team rule penalty records as follows:

Scores	Team member 1	Team member 2	Team member 3	Team member 4
Name				
Student ID				
Total mark				
Notes and Comments				

7 Project Management

Provide a Gantt chart showing the progress of your work here.

- Elements to be included: • Dates. • Tasks. • Bars. • Milestones. • Arrows (predecessors). • Task ID. • Resources. • Durations.
- Provide the slack time of each task
- Each primary task should have sub-tasks and each should be assigned to/accomplished by an individual.

Discuss the how well project management went.

- What were the keys to success?
- What do you think your team could do better next time?

8 Conclusions

- A summary of what you achieved. Mention all the design functions and objectives that you achieved while satisfying the constrains.
- Summarizes the key points of what you did (steps along the way and final outcome), and brings the document to a close.
- What did you learn from this experience?
- Not a word-for-word repetition of earlier material

9

10 References

- Use the IEEE reference style.
- Do not put in any references that are not cited in the text.

11 Appendix

- If you want to provide any additional information, use this appendix.
- Include data that is not necessary to an immediate understanding of the discussion.
- Appendices can contain drawings, calculations, technical specifications, or other detailed explanations of some aspects of your report.